



УДК 004.89

DOI <https://doi.org/10.38161/1996-3440-2024-2-71-82>

В. С. Федорова, В. В. Стригунов

РЕШЕНИЕ ОДНОЙ ЗАДАЧИ ОБНАРУЖЕНИЯ АНОМАЛИЙ СЕТЕВОГО ТРАФИКА С ПОМОЩЬЮ СВЕРТОЧНОЙ НЕЙРОННОЙ СЕТИ

Федорова В. С. – продуктовый аналитик СКБ Контур, e-mail: valerifedva@yandex.ru;
Стригунов В. В. – канд. физ.-мат. наук, доцент кафедры математических методов защиты информации и компьютерной безопасности, ТОГУ, e-mail: 005733@pnu.edu.ru

Обнаружение сетевых атак методом поиска аномалий заключается в выявлении моделей поведения, которые отклоняются от установленных базовых параметров, сигнализируя о потенциальных инцидентах безопасности. В статье рассмотрено применение сверточной нейронной сети для обнаружения аномалий сетевого трафика. В рамках исследования разработана сверточная нейронная сеть, проведено ее обучение на наборе данных набор данных CICIDS2017 и оценка качества. На основе разработанной нейронной сети построен прототип для детекции аномалий в сетевом трафике. Проведено тестирование и оценка качества прототипа на наборе данных CSE-CICIDS2018.

Ключевые слова: сетевой трафик, сетевая атака, обнаружение аномалий, сверточная нейронная сеть, набор данных.

Введение и постановка задачи

Повсеместное распространение компьютерных сетей и увеличивающееся количество угроз приводят к необходимости принятия надежных мер обеспечения информационной безопасности. В последние годы характер и изощренность киберугроз значительно изменились, перейдя от простых вторжений к сложным и скрытым атакам. Обнаружение сетевых атак осуществляется четырьмя типами методов [1]: на основе сигнатур, на основе аномалий, на основе спецификаций и гибридными методами. Наиболее распространенными являются первые два типа (на основе сигнатур и на основе аномалий), поэтому часто исследователи выделяют именно их [2–4]. Сигнатурный метод сводится к

поиску признаков уже известных атак по хранящимся в базе данных сигнатурам и шаблонам атак. Метод поиска аномалий заключается в выявлении моделей поведения, которые отклоняются от установленных базовых параметров, сигнализируя о потенциальных инцидентах безопасности. Раннее обнаружение аномалий позволяет принять превентивные меры, смягчить последствия атак и предотвратить несанкционированный доступ, утечку данных или сбой в работе сервисов.

С развитием и совершенствованием методов искусственного интеллекта они стали активнее использоваться в области информационной безопасности для защиты от новых угроз. Применение методов машинного и глубокого обучения для классификации сетевого трафика как аномального или нормального позволяет принимать решения в неопределенных ситуациях. Авторами отмечается [5], что данные методы являются наиболее перспективными и актуальными для изучения в рамках решения проблемы обнаружения аномалий.

Обобщенный алгоритм обнаружения вторжений на основе методов машинного обучения (ML, machine learning) или глубокого обучения (DL, deep learning) включает в себя выполнение трех основных этапов [4]: этап предварительной обработки данных, этап обучения и этап тестирования (рис. 1).

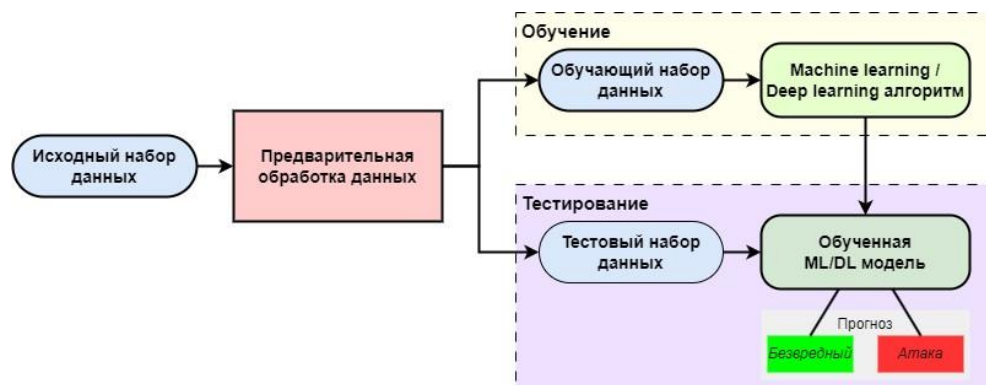


Рис. 1. Алгоритм обнаружения вторжений на основе методов машинного обучения

В последнее время вопросы, связанные с применением ML и DL для обнаружения атак, находятся в фокусе внимания исследователей. Так, в [6] рассмотрены вопросы построения и практической реализации системы обнаружения веб-атак на основе модели машинного обучения «случайный лес» (RF) и набора данных CICIDS2017. В работах [7, 8] в качестве структуры нейронной сети для решения поставленной задачи применялся многослойный персептрон, а в качестве набора данных выбран NSL-KDD. В статье [9] рассматривается методика обнаружения аномалий и кибератак на основе интеграции методов фрактального анализа и машинного обучения. В [10] предложена схема



обнаружения вторжений с использованием технологии объединенного машинного обучения (IDS-FMLT) для обнаружения вторжений в гетерогенную сеть, состоящую из различных исходных сетей. В качестве наборов данных в указанной работе использовались KDD, CUP-99, NetML-2020.

Целью настоящей работы является разработка, обучение и оценка качества сверточной нейронной сети, а также построение на ее базе прототипа для детекции аномалий в сетевом трафике. В качестве программного инструментария выбраны язык программирования Python, библиотеки pandas, keras, TensorFlow, NumPy, scikit-learn, среда разработки Google Colaboratory, генератор и анализатор потоков сетевого трафика CICFlowMeter [11].

Подготовка исходного набора данных

Для решения поставленной задачи выбран набор данных CICIDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017), подготовленный в Канадском институте кибербезопасности для исследования систем обнаружения вторжений [12, 13]. Общий объем подготовленных в наборе данных составляет более 50 Гб, а обработанные CSV-файлы содержат 2 830 540 записей, из которых 2 359 087 относятся к безвредному трафику, а 471 453 к трафику атак. Подробное распределение количества записей по классам атак представлено в табл. 1.

Таблица 1

Типы и количество записей в наборе данных CICIDS2017

Тип записи	Количество записей
Benign	2359087
DoS Hulk	231072
PortScan	158930
DDos	41835
DoS GoldenEye	10293
FTP Patator	7938
SSH-Patator	5897
DoS-slowloris	5796
DoS Slowhttptest	5499
Bot	1966
Web Attack – Brute Forse	1507
Web Attack – XSS	652
Infiltration	36
Web Attack – SQL Injection	21
Heartbleed	11

Для начала была выполнена конкатенация всех восьми CSV-файлов набора данных CICIDS2017. Так как целью решения задачи является обнаружение аномального трафика, то разновидность аномалий не так важна. Поэтому все записи, за исключением типа «безвредный», были объединены в единый класс «атака». Как показано выше, 2 359 087 записей относятся к безвредному трафику, а 471 453 – к трафику атак, что говорит о несбалансированности данных. Несбалансированность данных негативно сказывается на работе нейронных сетей – алгоритм игнорирует малочисленный класс, что приводит к плохому результату классификации [14]. Поэтому далее методом уменьшения выборки преобладающего класса (Undersampling) была произведена балансировка, чтобы нейросеть лучше знала вредоносный трафик и не переобучилась на нормальном трафике, что критически важно для достоверности результатов (рис. 2).

```
from imblearn.under_sampling import RandomUnderSampler

# делим датафрейм на признаки и целевое значение
x = df.iloc[:, df.columns != 'Label']
y = df[['Label']]
# Создание экземпляра RandomUnderSampler
rus = RandomUnderSampler()
# Применение уменьшения выборки к данным
x, y = rus.fit_resample(x, y)
df = x
df['Label'] = y
df.to_csv('/content/drive/My Drive/CICIDS2017/MachineLearningCVE/MachineLearningCVE1.csv')
```

Рис. 2. Программный код балансировки классов

Дополнительно была произведена подготовка данных для обучения путем замены текстовых значений классов на числовые с использованием механизма класса `LabelEncoder` из библиотеки `sklearn`. Этот шаг стандартизирует представление данных, сделав их пригодными для обучения машинной модели.

В рамках оптимизации данных проведено удаление дубликатов, выбросов и пустых значений, что способствует улучшению качества и консистентности данных. Таким образом, обеспечивается надежность и точность в дальнейшем анализе, а также исключается влияние возможных аномалий. Программный код оптимизации данных представлен на рис. 3.

Окончательный этап предобработки данных заключается в разделении набора данных на обучающую и тестовую выборки (рис. 4). Это обеспечивает возможность эффективной оценки производительности модели и гарантирует ее генерализацию на новые данные.



```
def _process(df: pd.DataFrame) -> (np.ndarray, np.ndarray):
    # Заполнение NaN средним значением для каждого класса в этом наборе данных
    nan_rows = df[df.isna().any(axis=1)].shape[0]
    logging.info("Fill NaN in {} rows with average value of each class.".format(nan_rows))
    df.iloc[:, df.columns != "Label"] = df.groupby("Label").transform(lambda x: x.fillna(x.mean()))

    # Замена значения inf на максимальное значение для каждого класса
    inf_rows = df[df.isin([np.inf]).any(axis=1)].shape[0]
    logging.info("Replace Inf in {} rows with maximum value of each class.".format(inf_rows))
    # Временная замена inf на NaN
    df = df.replace([np.inf], np.nan)
    # Замена inf максимальным значением для каждого класса в этом наборе данных
    df.iloc[:, df.columns != "Label"] = df.groupby("Label").transform(lambda x: x.fillna(x.max()))

    # Замена отрицательного значения минимальным положительным значением каждого класса
    logging.info("Replace negative values with minimum value of each class.")
    # Временная замена отрицательного значения на NaN
    df[df < 0] = np.nan
    # Замена отрицательного значения минимальным значением для каждого класса в этом наборе данных
    df.iloc[:, df.columns != "Label"] = df.groupby("Label").transform(lambda x: x.fillna(x.min()))

    return df
```

Рис. 3. Программный код оптимизации данных

```
def _split_train_test(df: pd.DataFrame) -> (pd.DataFrame, pd.DataFrame):
    # разделение датасета на признаки и целевые значения
    x = df.iloc[:, df.columns != 'Label']
    y = df[['Label']]
    # разделение на тестовую и обучающую выборки
    x_train, x_test, y_train, y_test = train_test_split(x, y, stratify=y, test_size=0.20,
                                                         random_state=np.random.randint(10))

    del x, y

    train = pd.concat([x_train, y_train], axis=1, sort=False)
    test = pd.concat([x_test, y_test], axis=1, sort=False)

    return train, test
```

Рис. 4. Программный код деления набора данных на обучающую и тестовую выборки

В результате выполненных действий набор данных CICIDS2017 приобрел форму, пригодную для последующего использования в исследовательском анализе, обучении моделей ML и проверке их эффективности в области кибербезопасности.

Разработка и обучение нейронной сети

Для решения поставленной задачи в качестве архитектуры нейронной сети была выбрана модель сверточной нейронной сети (CNN), в основе которой лежит принцип извлечения иерархии признаков из входных данных. Рассмотрим подробнее схему используемой нейронной сети (рис. 5).

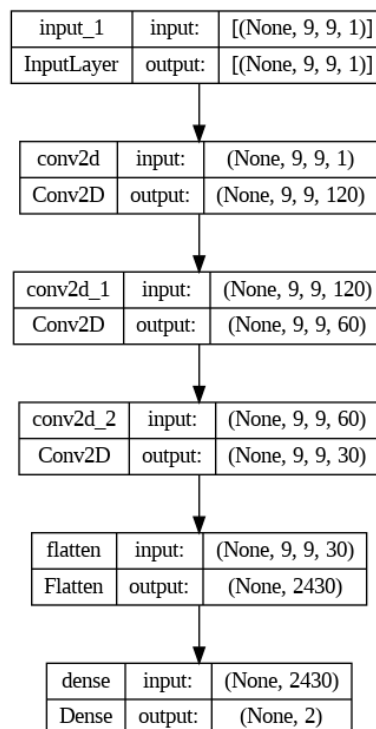


Рис. 5. Схема нейронной сети

Входной слой (Input Layer). В начале процесса входные данные, представляющие собой матрицы размером 9×9 с одним каналом, поступают на этот слой. Он принимает и структурирует их для дальнейшей обработки.

Сверточные слои (Convolutional Layers). Следом за входным слоем идут три сверточных слоя. Каждый такой слой использует фильтры (ядра) различного размера для сканирования входных данных и выделения специфичных признаков. Для введения нелинейности и акцентирования значимых аспектов данных применяется активационная функция ReLU [15].



Сглаживание (Flatten). После прохождения через сверточные слои, данные проходят через слой сглаживания, который преобразует трехмерные данные в одномерные. Это необходимо для передачи данных в следующий полносвязный слой.

Полносвязный слой (Dense Layer). Полносвязный слой принимает одномерные данные и выполняет классификацию на 2 класса, применяя функцию активации softmax. Эта функция преобразует выходные значения в вероятности принадлежности входных данных к каждому из классов.

На рис. 6 представлен программный код, реализующий нейросеть. После определения архитектуры модель компилируется с использованием функции потерь 'sparse_categorical_crossentropy', оптимизатора 'adam' и метрики 'sparse_categorical_accuracy'.

```
def create_cnn_model() -> keras.Model:
    # создание слоев
    inputs = keras.layers.Input(shape=(9, 9, 1))
    x = keras.layers.Conv2D(120, 2, activation='relu', padding="same")(inputs)
    x = keras.layers.Conv2D(60, 3, activation='relu', padding="same")(x)
    x = keras.layers.Conv2D(30, 4, activation='relu', padding="same")(x)
    x = keras.layers.Flatten()(x)
    outputs = keras.layers.Dense(2, activation='softmax')(x)
    cnn_model = keras.Model(inputs=inputs, outputs=outputs, name='cnn')

    # Компиляция слоев
    cnn_model.compile(loss='sparse_categorical_crossentropy',
                      metrics=['sparse_categorical_accuracy'],
                      optimizer='adam')

    return cnn_model

model = create_cnn_model()
```

Рис. 6. Программный код разработки нейронной сети

Процесс обучения проводился на тренировочной выборке с применением алгоритма обратного распространения ошибки. Веса модели постепенно корректировались в процессе минимизации функции потерь, что обеспечивало сходимость модели к оптимальным параметрам. Контрольные точки сохранения позволяли отслеживать прогресс обучения и обеспечивать возможность восстановления модели при необходимости.

Тестирование и оценка качества нейронной сети

Разработанная сверточная нейронная сеть подверглась процедуре валидации на отложенной части данных (тестовый набор данных), которые не использовались в процессе обучения. В качестве метрик для оценки выбраны следующие четыре [16].

Доля правильных ответов

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN},$$

где TP – количество верно предсказанных положительных классов; TN – количество верно предсказанных отрицательных классов; FP – количество ложно положительных предсказаний; FN – количество ложно отрицательных предсказаний.

В контексте обнаружения аномалий, где важно как верное обнаружение аномалий, так и минимизация ложных срабатываний, ассигасу дает общую картину производительности модели. Однако, следует отметить, что в случае несбалансированных классов, ассигасу может быть искаженной метрикой, и поэтому ее следует анализировать в сочетании с другими метриками.

Точность

$$precision = \frac{TP}{TP + FP},$$

где TP – верно предсказанные положительные классы; FP – ложно положительные предсказания.

Точность является важной метрикой в контексте обнаружения аномалий, поскольку она предоставляет общий процент правильных предсказаний модели. В задачах без явного доминирования класса аномалий точность дает нам общую картину того, насколько хорошо модель справляется с обнаружением как нормальных, так и аномальных паттернов в сетевом трафике.

Полнота

$$recall = \frac{TP}{TP + FN},$$

где TP – верно предсказанные положительные классы; FN – ложно отрицательные предсказания.

Полнота является критической метрикой в задачах детекции аномалий, так как она измеряет способность модели обнаруживать все реальные аномалии в сетевом трафике. Снижение полноты может привести к пропуску важных аномалий, что недопустимо в контексте безопасности сети. Поэтому высокая полнота является ключевым критерием для эффективной детекции аномалий.

F1-мера

$$F1 = \frac{2 \cdot precision \cdot recall}{precision + recall}.$$

Эта метрика особенно полезна в случаях, когда нужно учесть как точность, так и полноту модели. В контексте обнаружения аномалий, где ложные срабатывания и пропуски аномалий могут иметь серьезные последствия, использование F1-меры позволяет достичь компромисса между точностью и полнотой.

Рассмотрим результаты проведенной оценки (рис. 7). Здесь класс 0 – аномальный трафик, класс 1 – безвредный.



```
Loss: 0.08212973177433014
Accuracy : 0.9480810165405273

09:19:54 INFO
      precision    recall  f1-score   support

     0         1.00      0.90      0.95     223058
     1         0.91      1.00      0.95     223060

   accuracy          0.95
  macro avg          0.95
 weighted avg          0.95
```

Рис. 7. Результат оценки качества нейронной сети

Разработанная модель нейронной сети дает хорошие результаты оценки с высокой точностью, полнотой и F1-мерой для обоих классов. Видно, что точность предсказаний для класса аномальный трафик составляет 100%, полнота 90%, а F1-мера 95%. Для класса безвредного трафика точность также высока и составляет 91%, а полнота 100%, что свидетельствует о хорошей способности модели выявлять положительные примеры. В целом, доля правильных ответов (ассигасу) нейронной сети составляет 95%, что подтверждает ее качество и эффективность.

Создание и тестирование прототипа детектора аномалий

Для того чтобы оценить практическую эффективность разработанной нейронной сети в контексте данных, отличных от тех, на которых она была обучена, необходимо создать прототип детектора аномалий сетевого трафика. Такой прототип должен быть приближен к реальному инструменту для обнаружения аномалий, поэтому на вход ему будут подаваться PCAP-файлы. PCAP-файлы содержат информацию о переданных сетевых пакетах, они используются для сохранения и анализа трафика сети. Из имеющихся открытых наборов PCAP-файлов выбран CSE-CIC-IDS2018 [17], подготовленный в рамках совместного проекта Центра безопасности коммуникаций (CSE, Канада) и Канадского института кибербезопасности (CIC). Он содержит в себе множество PCAP файлов с безвредным и вредоносным трафиком, в том числе содержит аномалии, которых не было в обучающих данных.

Логика функционирования прототипа включает в себя несколько этапов. Первый – последовательная обработка PCAP-файлов при помощи генератора и анализатора потоков сетевого трафика CICFlowMeter [11]. Данный инструмент проводит анализ сетевого трафика и формирует выходной файл в формате CSV требуемой структуры. Следующим этапом является передача CSV-файла в нейронную сеть, результатом работы которой является файл, указывающий принадлежность каждого потока сетевых пакетов к определенному классу – «безвредный» или «атака». Схема работы прототипа приведена на

рис. 8. Так как рассматривается лишь прототип, то передача данных между инструментами, обозначенная стрелками, производится вручную. При необходимости ее можно автоматизировать.

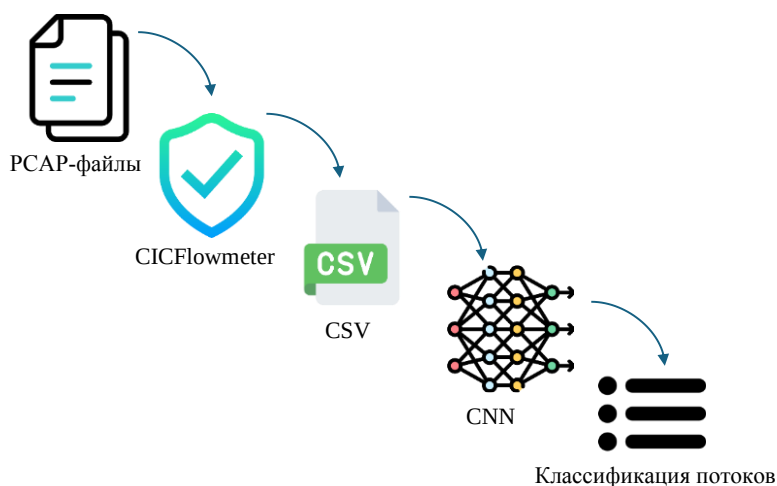


Рис. 8. Схема прототипа детектора аномалий

После успешного тестирования была произведена оценка качества прототипа на основе разработанной сверточной нейронной сети на реальных данных. Результаты представлены на рис. 9.

	precision	recall	f1-score	support
0	0.81	0.76	0.78	50104
1	0.86	0.90	0.88	86829
accuracy			0.85	136933
macro avg	0.84	0.83	0.83	136933
weighted avg	0.85	0.85	0.85	136933

Рис. 9. Оценка качества прототипа

Полученный результат показывает, что разработанная нейронная сеть хорошо справляется с задачей классификации, обнаруживая как аномальный, так и безвредный трафик с высокой точностью. При этом она демонстрирует надежные показатели по precision (точность) и recall (полнота) для обоих классов. Важно отметить, что достигнутая точность на уровне 85% говорит о том, что модель успешно учла разнообразие данных и обобщила шаблоны, что делает ее надежной в реальных условиях.



Библиографические ссылки

1. A comprehensive survey on network anomaly detection / Fernandes G., Rodrigues J., Carvalho L. F., Al-Muhtadi J. F., Proença M. L. // *Telecommunication Systems: Modelling, Analysis, Design and Management*. 2019. Vol. 70, № 3. P. 447-489.
2. Сапожников А. А. Обнаружение аномальной сетевой активности // *Доклады ТУСУР*. 2009. Т. 2, № 1. С. 79-80.
3. Нейросетевая технология обнаружения сетевых атак на информационные ресурсы / Емельянова Ю. Г., Талалаев А. А., Тищенко И. П., Фраленко В. П. // *Программные системы: теория и приложения*. 2011. Т. 2, № 3. С. 3-15.
4. Network intrusion detection system: A systematic study of machine learning and deep learning approaches / Zeeshan A., Adnan S. K., Cheah W. S., Johari A., Farhan A. // *Transactions on Emerging Telecommunications Technologies*. 2021. № 32.
5. Обнаружение аномалий сетевого трафика: основные аспекты, проблемы и методы / Будько М. Б., Малько А. Д., Стародубова Д. Д., Стародубов Р. Д. // *Современная наука: актуальные проблемы теории и практики. Сер.: естественные и технические науки*. 2020. № 8. С. 46-49.
6. Горюнов М. Н., Мацкевич А. Г., Рыболовлев Д. А. Синтез модели машинного обучения для обнаружения компьютерных атак на основе набора данных CICIDS2017 // *Труды ИСП РАН*. 2020. Т. 32, № 5. С. 81-93.
7. Татарникова Т. М., Бимбетов Ф., Богданов П. Ю. Выявление аномалий сетевого трафика методом глубокого обучения // *Известия СПбГЭТУ «ЛЭТИ»*. 2021. № 4. С. 36-41.
8. Зуев В.Н. Обнаружение аномалий сетевого трафика методом глубокого обучения // *Программные продукты и системы*. 2021. Т. 34, № 1. С. 91-97.
9. Методика обнаружения аномалий и кибератак на основе интеграции методов фрактального анализа и машинного обучения / Котенко И. В., Саенко И. Б., Лаута О. С., Крибель А. М. // *Информатика и автоматизация*. 2022. Т. 21, № 6. С. 1328-1358.
10. A Fused Machine Learning Approach for Intrusion Detection System/ Farooq M. S., Abbas S., Sultan K., Atta-ur-Rahman M. A., Khan M.A., Mosavi A. // *Computers, Materials & Continua*. 2023. Vol. 74, № 2. P. 2607-2623.
11. CICFlowMeter. Canadian Institute for Cybersecurity. URL: <https://www.unb.ca/cic/research/applications.html> (дата обращения: 01.03.2024).
12. Intrusion detection evaluation dataset (CIC-IDS2017). Canadian Institute for Cybersecurity. URL: <https://www.unb.ca/cic/datasets/ids-2017.html> (дата обращения: 01.03.2024).



13. Sharafaldin I., Lashkari A. H., Ghorbani A. A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. // ICISp. 2018. Т. 1. С. 108-116.
14. Никулин В. Н., Канищев И. С., Багаев И. В. Методы балансировки и нормализации данных для улучшения качества классификации // Компьютерные инструменты в образовании. 2016. № 3. С. 16-24.
15. Особенности работы сверточных нейронных сетей / Скрипачев В. О., Гуйда М. В., Гуйда Н. В., Жуков А. О. // International Journal of Open Information Technologies. 2022. Т. 10, № 12. С. 53-61.
16. Михайличенко А. А. Аналитический обзор методов оценки качества алгоритмов классификации в задачах машинного обучения // Вестник Адыгейского государственного университета. Сер. 4: Естественно-математические и технические науки. 2022. № 4. С. 52-59.
17. A Realistic Cyber Defense Dataset (CSE-CIC-IDS2018). Registry of Open Data on AWS. URL: <https://registry.opendata.aws/cse-cic-ids2018/> (дата обращения: 01.03.2024).

Title: Solving One Problem of Network Traffic Anomaly Detection Using a Convolutional Neural Network

Authors' affiliation:

Fedorova V. S. – SKB Kontur, Khabarovsk, Russian Federation

Strigunov V. V. – Pacific National University, Khabarovsk, Russian Federation

Abstract: Detecting network attacks by anomaly search method is to identify behaviors that deviate from established baseline parameters, signaling potential security incidents. In this paper, the authors consider the application of convolutional neural network for network traffic anomaly detection. As part of the study, a convolutional neural network has been developed, trained on the dataset CICIDS2017 dataset and quality assessment has been carried out. Based on the developed neural network, a prototype for anomaly detection in network traffic has been built. Testing and quality assessment of the prototype on the CSE-CIC-IDS2018 dataset has been performed.

Keywords: network traffic, network attack, anomaly detection, convolutional neural network, dataset.